

Formal Verification of the Otoka Crossroad

Step 1: Labeled Transition System (LTS) & Formal State Verification

1. System Definition and State Mapping

Before we can prove a system is safe, we must mathematically define its states. A state represents the exact condition the system is in at any given moment.

For the Otoka Crossroad, the system is designed as a Synchronous Moore Finite State Machine (FSM). The physical crossroad consists of two main intersecting axes:

- **East-West (EW):** Bulevar Meše Selimovića (Main Boulevard)
- **North-South (NS):** Buća Potok & Švrakino Selo (Side Streets)

We denote our set of states as $Proc = \{S_0, S_1, S_2, S_3\}$. Each state is defined as an ordered pair representing the physical traffic lights: $(Light_{EW}, Light_{NS})$.

- **$S_0 = (G_{EW}, R_{NS})$:** East-West is Green, North-South is Red. *(Meaning: Cars on the main boulevard are driving freely. Cars from Buća Potok are stopped.)*
- **$S_1 = (Y_{EW}, R_{NS})$:** East-West is Yellow, North-South is Red. *(Meaning: The main boulevard is warned to slow down and prepare to stop. Side streets are still safely held at a red light.)*
- **$S_2 = (R_{EW}, G_{NS})$:** East-West is Red, North-South is Green. *(Meaning: The main boulevard is completely stopped. Cars from the side streets/bridge are now driving safely.)*
- **$S_3 = (R_{EW}, Y_{NS})$:** East-West is Red, North-South is Yellow. *(Meaning: The side streets are warned to slow down, while the main boulevard remains stopped.)*

2. Formal LTS Definition

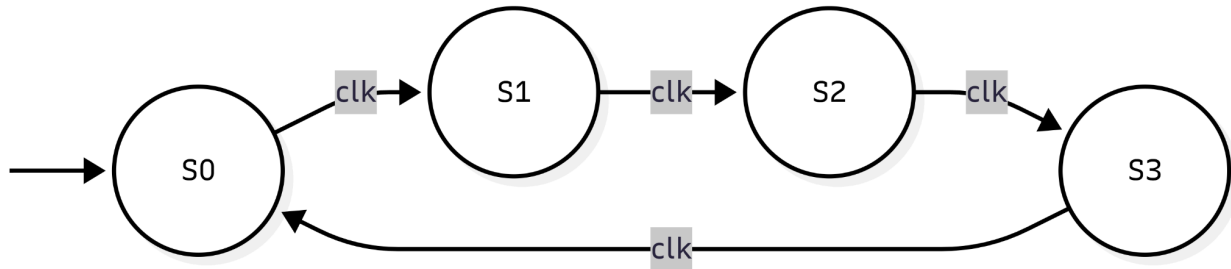
An LTS describes how a system moves from one state to another. It is mathematically defined as a tuple $(Proc, Act, \rightarrow)$:

- **Proc (Set of States):** $\{S_0, S_1, S_2, S_3\}$
- **Act (Set of Actions):** $\{clk\}$. In our physical PCB hardware, the transitions are not random; they are strictly triggered by the rising edge of the 555-timer clock signal feeding into the 74HCT74 D-Type Flip-Flops. Therefore, the only action that causes a state change is a

clock pulse (clk).

- $\rightarrow$ (**Transition Relation**): The set of valid state-pairs, mapping how the system evolves over time.

$\rightarrow = \{ (S_0, \text{clk}, S_1), (S_1, \text{clk}, S_2), (S_2, \text{clk}, S_3), (S_3, \text{clk}, S_0) \}$



3. Formal Proofs (Manual Model Checking)

We require checking properties like Reachability, Deadlocks, Fairness, Liveness, and Safety. We will now prove these mathematically using our LTS model to guarantee that car crashes cannot happen at Otoka.

Proof 1: Is every state reachable?

- **Question:** Can the hardware actually execute every traffic phase, or are some lights permanently broken/skipped?
- **Proof:** A state s_n is reachable from the initial state S_0 if there exists a valid trace sequence.
 1. The system powers on at S_0 .
 2. $S_0 \rightarrow S_1$ via clk & implies; S_1 is reachable.
 3. $S_1 \rightarrow S_2$ via clk & implies; S_2 is reachable.
 4. $S_2 \rightarrow S_3$ via clk & implies; S_3 is reachable.
- **Conclusion:** The reachability set is $\{S_0, S_1, S_2, S_3\}$. **Yes, every state is reachable.** The physical controller successfully cycles through all designed traffic phases.

Proof 2: Are there deadlock states?

- **Question:** Will the intersection ever "freeze" (e.g., getting stuck on a red light forever, causing massive traffic jams)?
- **Proof:** A deadlock occurs if the system enters a state s where no further actions are possible (an out-degree of 0). Looking at our transition relation $\rightarrow$, every single state has exactly one outgoing transition triggered by the clk action. Mathematically, $\forall s \in \text{Proc}, \exists s' \in \text{Proc}$ such that $s \rightarrow s'$ via clk.
- **Conclusion:** Because no state lacks an outgoing arrow, the intersection will always progress to the next phase upon a clock pulse. **There are no deadlock states.**

Proof 3: Does the system return to the initial state? Are there loops?

- **Question:** After one full traffic cycle, does the system reset correctly to start again?
- **Proof:** Tracing the transitions from S_0 , we follow the path: $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_0$.
- **Conclusion:** **Yes**, after exactly 4 clock pulses, the system returns perfectly to S_0 . **Yes**, there is an infinite loop, which is the strictly desired behavior for a continuous reactive system like a traffic light.

Proof 4: Liveness and Fairness

- **Question:** If a car arrives on the Buća Potok side street, will it eventually get a green light, or will the main boulevard hoard the green light forever?
- **Proof:** Fairness dictates that no process (or street) is starved of resources. Liveness means "something good eventually happens." Because our LTS is a single, deterministic loop without branching or deadlocks, the trace from any state will *always* eventually reach S_2 (North-South Green). For instance, if a car arrives during S_0 , the system *must* execute the trace $\langle \text{clk}, \text{clk} \rangle$ to reach S_2 .
- **Conclusion:** The system is **strictly fair and guarantees liveness**. A car waiting at any red light is mathematically guaranteed to receive a green light within a finite number of clock cycles.

Proof 5: Is the system Safe? (Crash Avoidance)

- **Question:** Can car crashes happen? Are there conflicting lights?
- **Proof:** In formal models, Safety means "something bad never happens." For a crossroad, a "bad state" is defined as:
 1. Overlapping Green lights (G_{EW} and G_{NS} active simultaneously).
 2. Abrupt light changes (Green directly to Red without a Yellow warning).

Checking for Overlapping Greens: Let's inspect our set Proc. S_0 has G_{EW} . S_2 has G_{NS} . There is no state in Proc defined as (G_{EW}, G_{NS}) . Since an FSM can only occupy one state at any given moment, it is mathematically impossible for both axes to be green.

Checking for Abrupt Changes:

$S_0 (G_{EW}) \rightarrow S_1 (Y_{EW}) \rightarrow S_2 (R_{EW})$.

$S_2 (G_{NS}) \rightarrow S_3 (Y_{NS}) \rightarrow S_0 (R_{NS})$.

Every green light strictly transitions to a yellow light before turning red.

- **Conclusion:** **The system is absolutely safe.** The definition of the states and the transition relations fundamentally prevent hardware-level car crashes.

Step 2: Petri Net Modeling and

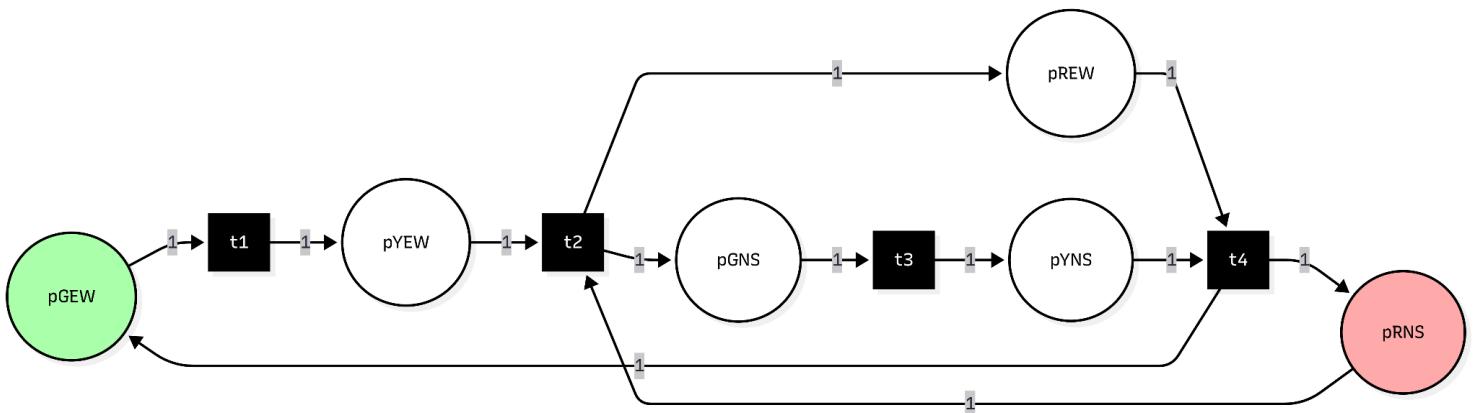
Behavioral Analysis

1. Translating the Physical Crossroad into a Petri Net

A Petri Net is a mathematical model used to prove that concurrent systems share resources safely. In our physical Otoka crossroad, the "resource" is the electrical activation signal coming from Flip-Flops. If the electricity is routed to the wrong lights at the wrong time, cars crash. We model this to prove it is structurally impossible.

- **Places (P):** These represent the physical LED traffic lights on your PCB. We have 6 places: p_{GEW} , p_{YEW} , p_{REW} , p_{GNS} , p_{YNS} , p_{RNS} .
- **Transitions (T):** These represent the electrical clock pulses from the 555 timer. A transition firing means a clock pulse has forced the Flip-Flops to change state. We have 4 transitions: t_1 , t_2 , t_3 , t_4 .
- **Tokens & Initial Marking (M_0):** A "token" (a black dot inside a place) means that specific light currently has power and is ON. When the system powers up in State S_0 , the main boulevard is Green and the side street is Red. Therefore, we place exactly 1 token in GEW and 1 token in RNS.

$$M_0 = (1, 0, 0, 0, 0, 1)^T$$



2. Mathematical Matrix Representation

To rigorously prove the safety properties, we convert the graphical token flow into matrices. The rows are our 6 places (lights), and the columns are our 4 transitions (clock ticks).

Pre-Matrix (What is consumed to trigger a change):

Look at column t_2 : To transition from S_1 to S_2 , the system must "consume" the Yellow light on the boulevard and the Red light on the side street. Hence, there are 1s at YEW and RNS for t_2 .

Pre =

$| 1 \ 0 \ 0 \ 0 |$ (GEW)
 $| 0 \ 1 \ 0 \ 0 |$ (YEW)
 $| 0 \ 0 \ 0 \ 1 |$ (REW)
 $| 0 \ 0 \ 1 \ 0 |$ (GNS)
 $| 0 \ 0 \ 0 \ 1 |$ (YNS)
 $| 0 \ 1 \ 0 \ 0 |$ (RNS)

Post-Matrix (What is produced after the change):

Look at column t_2 again: After the clock ticks, the system "produces" a Red light on the boulevard and a Green light on the side street.

Post =

$| 0 \ 0 \ 0 \ 1 |$ (GEW)
 $| 1 \ 0 \ 0 \ 0 |$ (YEW)
 $| 0 \ 1 \ 0 \ 0 |$ (REW)
 $| 0 \ 1 \ 0 \ 0 |$ (GNS)
 $| 0 \ 0 \ 1 \ 0 |$ (YNS)
 $| 0 \ 0 \ 0 \ 1 |$ (RNS)

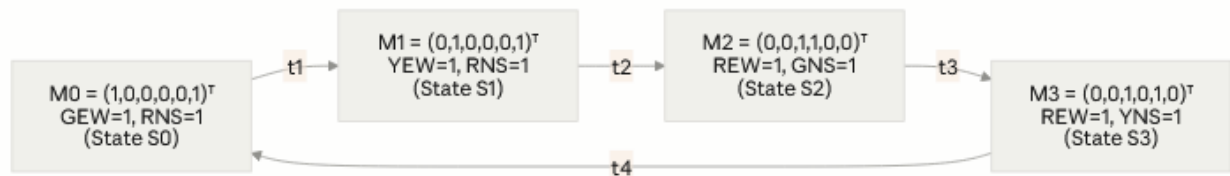
Incidence Matrix (C = Post - Pre):

This matrix represents the net change in tokens for every clock tick.

C =

$| -1 \ 0 \ 0 \ 1 |$
 $| 1 \ -1 \ 0 \ 0 |$
 $| 0 \ 1 \ 0 \ -1 |$
 $| 0 \ 1 \ -1 \ 0 |$
 $| 0 \ 0 \ 1 \ -1 |$
 $| 0 \ -1 \ 0 \ 1 |$

2.1 Reachability Graph



Determinism of the Firing Sequence: At every reachable marking, exactly one transition is enabled, which is why the reachability graph above is a simple cycle rather than a branching tree. We verify this using the enabling rule: a transition t is enabled at marking M if and only if $\text{Pre}[:,t] \leq M$ (every place required by that transition already holds a token).

- At $M0 = (1,0,0,0,0,1)$: only $t1$'s precondition ($GEW \geq 1$) is satisfied $\rightarrow$ **$t1$ enabled**, no others.
- At $M1 = (0,1,0,0,0,1)$: only $t2$'s precondition ($YEW \geq 1, RNS \geq 1$) is satisfied $\rightarrow$ **$t2$**

enabled, no others.

- At **M2** = (0,0,1,1,0,0): only t3's precondition ($GNS \geq 1$) is satisfied → **t3 enabled**, no others.
- At **M3** = (0,0,1,0,1,0): only t4's precondition ($REW \geq 1, YNS \geq 1$) is satisfied → **t4 enabled**, no others.

Because no marking ever enables more than one transition, the net exhibits no conflict and no concurrency — the firing sequence $\sigma = t_1 t_2 t_3 t_4$ is the *unique* path from M_0 , which is precisely why the reachable set $R(N, M_0)$ contains exactly the 4 markings shown above and nothing else.

3. Proving the 4 Behavioral Properties

3.1 Reachability

Definition: A marking M' is reachable from M_0 if there exists a finite firing sequence $\sigma = t_{i_1} t_{i_2} \dots t_{i_k}$ such that $M_0 \rightarrow M'$.

Crossroad Proof: Starting from M_0 (State 0: GEW, RNS), the clock triggers transition t_1 , producing M_1 (State 1: YEW, RNS). The sequence strictly follows $\sigma = t_1 t_2 t_3 t_4$. Because the hardware logic gates strictly guide this sequence, the set of all reachable markings $R(N, M_0)$ contains exactly 4 predefined safe markings. No random or broken marking (like all lights turning off) is reachable.

3.2 Boundedness

Definition: A net is k -bounded if $M(p) \leq k$ for every place p . A 1-bounded net is called *safe*. It addresses whether a storage place will overflow.

Crossroad Proof: In our physical circuit, "overflow" would mean a single LED receiving a double dose of logic-high voltage, or the system attempting to activate multiple contradictory states. Because our token conservation (proven in 4.4) strictly limits the total system to 2 tokens, and the matrix structure never places more than 1 token in any single place, the maximum value of $M(p)$ for any light is exactly 1. **The net is 1-bounded (Safe).**

3.3 Liveness

Definition: A net is live if, from every reachable marking, every transition can eventually be fired. This guarantees deadlock-free operation.

Distinguishing Liveness from Deadlock-Freedom: These two properties are related but not equivalent, and it is important to state the difference precisely. A net is **deadlock-free** if every reachable marking has at least one enabled transition (out-degree ≥ 1) — the system never fully freezes, but individual transitions could still go permanently unused. A net is **live** if, from every

reachable marking, every transition can *eventually* be fired again — a strictly stronger guarantee that no single transition is ever starved forever, even if the system keeps moving.

In our crossroad, these two properties happen to coincide because the net has no conflict or concurrency (proven above): each marking enables exactly one transition, and the T-invariant $y = (1, 1, 1, 1)$ shows that firing sequence visits t_1 , t_2 , t_3 , and t_4 exactly once per cycle before returning to M_0 . This simultaneously proves deadlock-freedom (there is always exactly one enabled transition, so out-degree is never 0) and liveness (every transition fires infinitely often, once per cycle, forever) — but it is worth noting the two proofs are logically distinct even though the same T-invariant argument happens to establish both here.

Crossroad Proof: We mathematically prove this using a **T-Invariant**. By solving the equation $Cy = 0$, we find the minimum T-invariant vector $y = (1, 1, 1, 1)^T$. Because this vector contains all 1s (and no 0s), it mathematically proves that firing every transition exactly once (t_1, t_2, t_3, t_4) successfully returns the system to M_0 without getting stuck. Because it forms a perfect, unbreakable loop, from any given state, every transition will eventually fire. The intersection is perfectly **Live and Deadlock-free**.

3.4 Conservation and Strict Safety Proof (State Equation Analysis)

Definition: A net is conservative if the weighted token sum remains constant. Beyond conservation, we must formally prove that a "crash state" is mathematically unreachable using the fundamental Petri Net State Equation: $M = M_0 + Cn$, where n is the firing vector.

Crossroad Proof (The "No Car Crash" Guarantee):

By solving the null space of the Incidence Matrix $C^T x = 0$, we identify **two P-invariants** (of the three that span the net's full invariant space) that together prove independent resource bounding:

- P-Invariant 1 (East-West Bounding): $x_1 = (1, 1, 1, 0, 0, 0)^T$ proves that $M(\text{GEW}) + M(\text{YEW}) + M(\text{REW}) = 1$. This guarantees the main boulevard always has exactly one active light.
- P-Invariant 2 (North-South Bounding): $x_2 = (0, 0, 0, 1, 1, 1)^T$ proves that $M(\text{GNS}) + M(\text{YNS}) + M(\text{RNS}) = 1$. This guarantees the side street always has exactly one active light. Because both axes are strictly 1-bounded independently, duplicate active states on a single traffic pole are mathematically impossible.

Note on Completeness: The null space of C^T is in fact three-dimensional, meaning a third independent P-invariant also exists: $x_3 = (0, 0, 1, 0, 0, 1)^T$, proving $M(\text{REW}) + M(\text{RNS}) = 1$ — i.e., exactly one of "East-West is Red" or "North-South is Red" holds at every marking. This is a genuine structural property of the net, though it is not needed for the collision-safety argument below; x_1 and x_2 alone are sufficient to independently bound each axis to exactly one active light, which is what the crash-unreachability proof requires.

To definitively prove safety, we attempt to reach a crash state where both the main boulevard and the side street have Green lights simultaneously. We define this theoretical crash marking as $\mathbf{M}_{\text{crash}} = (1, 0, 0, 1, 0, 0)^T$ (Tokens in GEW and GNS).

Using the State Equation, we check if there is any integer firing sequence n that satisfies:

$$\mathbf{C} \cdot \mathbf{n} = \mathbf{M}_{\text{crash}} - \mathbf{M}_0$$

$$\mathbf{M}_{\text{crash}} - \mathbf{M}_0 = (1, 0, 0, 1, 0, 0)^T - (1, 0, 0, 0, 0, 1)^T = (0, 0, 0, 1, 0, -1)^T$$

We extract the system of linear equations from the rows of our Incidence Matrix (C) set equal to this difference:

1. (Row 1: GEW): $-n_1 + n_4 = 0 \Rightarrow n_1 = n_4$
2. (Row 2: YEW): $n_1 - n_2 = 0 \Rightarrow n_1 = n_2$
3. (Row 3: REW): $n_2 - n_4 = 0 \Rightarrow n_2 = n_4$
4. (Row 4: GNS): $n_2 - n_3 = 1$
5. (Row 5: YNS): $n_3 - n_4 = 0 \Rightarrow n_3 = n_4$
6. (Row 6: RNS): $-n_2 + n_4 = -1$

Resolution & Contradiction:

From equations 1, 2, 3, and 5, we establish that to maintain the flow of tokens, $n_1 = n_2 = n_3 = n_4$.

However, if we substitute $n_2 = n_4$ into equation 6, we get:

$$-(n_4) + n_4 = -1 \Rightarrow 0 = -1.$$

Similarly, substituting $n_2 = n_3$ into equation 4 yields:

$$n_3 - n_3 = 1 \Rightarrow 0 = 1.$$

Because this results in unsolvable mathematical contradictions, no valid firing vector n exists.

Step 3: Linear Temporal Logic (LTL) and Invariant Checking

1. LTL Fundamentals and System Mapping

Ordinary propositional logic is too rigid for traffic lights because it assigns a fixed truth value to a proposition. A traffic light changes over time. Linear Temporal Logic (LTL) repairs this by evaluating paths (sequences of states) over discrete time steps.

For our physical Otoka PCB, a "time step" corresponds to exactly one rising edge of the 555-timer clock pulse. We define our Atomic Propositions (AP) based directly on the outputs of the logic gates feeding the LEDs:

- **GEW:** Bulevar Meše Selimovića (Main) is Green.
- **YEW:** Bulevar Meše Selimovića (Main) is Yellow.
- **REW:** Bulevar Meše Selimovića (Main) is Red.
- **GNS:** Buća Potok (Side) is Green.
- **YNS:** Buća Potok (Side) is Yellow.
- **RNS:** Buća Potok (Side) is Red.

2. Formal Property Specification and Hardware-Level Proofs

We must rigorously translate our real-world safety requirements into LTL formulas. Because our system is a physical hardware FSM, our state is strictly defined by the 2-bit output of the 74HCT74 Dual D-Type Flip-Flop: (S_1 , S_0). All proofs will be grounded in the Boolean logic gates driving the LEDs.

Property 1: Absolute Safety (Collision Avoidance)

The Real-World Goal: Prove that a car crash caused by overlapping green lights is mathematically impossible at the logic-gate level.

LTL Formula: $G \neg(GEW \wedge GNS)$

Proof by Combinational Tautology (State Space Exhaustion):

Unlike complex software that requires step-by-step k-induction, our physical 2-bit state memory allows us to exhaustively prove this property across the entire state space:

1. Based on the 74HC08 AND gates in our schematic, the East-West Green light is active only when both flip-flops output 0. Therefore, $GEW = \neg S_1 \wedge \neg S_0$.
2. The North-South Green light is active only when S_1 is 1 and S_0 is 0. Therefore, $GNS = S_1 \wedge \neg S_0$.

3. We substitute these hardware equations into our LTL condition:

$$\neg((\neg S_1 \wedge \neg S_0) \wedge (S_1 \wedge \neg S_0))$$

4. By the fundamental laws of Boolean Algebra, $\neg S_1 \wedge S_1 = 0$ (a logical contradiction).

5. Therefore, the expression simplifies to $\neg(0)$, which equals **True** for all 2^2 possible states.

Conclusion: Because the physical logic gates require S_1 to simultaneously be 0 and 1, overlapping greens are mathematically and electrically impossible regardless of reachability. The safety property holds unconditionally.

2.1 Verification via Bounded Model Checking

As a complementary check to the tautology proof above, we verify $G \neg(GEW \wedge GNS)$ using Bounded Model Checking. Let the property be $P := \neg(GEW \wedge GNS)$, encoded over state bits

(S1, S0). The BMC query for bound k asks whether there exists a trace of length k, starting at the initial state and following valid transitions, that reaches a violation of P:

$$I(s_0) \wedge \bigwedge_{i=1}^k T(s_{i-1}, s_i) \wedge \neg P(s_k)$$

$I(s_0)$ initial state holds

$T(s_{i-1}, s_i)$ valid transition at each step, unrolled k times

$\neg P(s_k)$ the safety property is violated at step k

where $I(s_0) := (S1=0 \wedge S0=0)$, i.e. the system powers on at S0, and T is our transition relation $\rightarrow = \{(S0,S1),(S1,S2),(S2,S3),(S3,S0)\}$.

- **k=1:** $I(s_0) \wedge T(s_0,s_1) \wedge \neg P(s_1)$. The only reachable s1 from S0 is S1 (01). At S1, $GEW=0$, $GNS=0$, so $GEW \wedge GNS=0$, meaning $\neg P(s_1)=\neg(0)=0$ — **unsatisfiable**.
- **k=2:** $\dots \wedge T(s_1,s_2) \wedge \neg P(s_2)$. Reachable s2 = S2 (10). $GEW=0$, $GNS=1 \rightarrow GEW \wedge GNS=0 \rightarrow \neg P(s_2)=0$ — **unsatisfiable**.
- **k=3:** $\dots \wedge T(s_2,s_3) \wedge \neg P(s_3)$. Reachable s3 = S3 (11). $GEW=0$, $GNS=0 \rightarrow$ **unsatisfiable**.
- **k=4:** $\dots \wedge T(s_3,s_4) \wedge \neg P(s_4)$. Reachable s4 = S0 (00) again. $GEW=1$, $GNS=0 \rightarrow$ **unsatisfiable**.

Because the system is a deterministic 4-cycle, k=4 already covers every reachable state exactly once, and the query remains unsatisfiable at every bound. Since no counterexample trace exists at any k — including as $k \rightarrow \infty$, by the cyclic repetition — the BMC procedure confirms **$G\neg(GEW \wedge GNS)$ holds unconditionally**.

Both methods reach the same conclusion by different routes: the tautology proof shows the violation is impossible at the logic-gate level regardless of reachability, while BMC confirms no reachable trace ever exhibits it — together demonstrating that the safety property is robust both structurally and behaviorally.

Property 2: Liveness (No Traffic Starvation)

The Real-World Goal: Guarantee that if a car pulls up to a red light at Buća Potok (Side Street), it will eventually get a green light.

LTL Formula: $G(RNS \Rightarrow F(GNS))$

Proof by State-Bit Tracing and Hardware Fairness:

(Fairness Assumption: This proof assumes the 555-timer continuously provides clock pulses, satisfying the LTL requirement for infinite paths).

Assume the system is currently in a state where RNS is True. According to our truth table, RNS is active when the state bits are **00** or **01**.

1. Because our 74HCT86 (XOR) and 74HCT04 (NOT) gates act as a strict 2-bit up-counter, a clock pulse unconditionally forces the bits to transition: **00** $\rightarrow$ **01** $\rightarrow$ **10**.
2. In state **10**, the logic evaluates to GNS = True ($S_1=1$, $S_0=0$).
3. Because the flip-flop Set (S) and Reset (R) pins are tied inactive in the schematic, no asynchronous interruptions can halt the D-flip-flop sequence. Reaching state 10 is structurally inevitable.

Conclusion: F(GNS) evaluates to True. Liveness is mathematically guaranteed by the continuous counter architecture.

Property 3: Strict Sequencing (Hardware-Level Phase Changes)

The Real-World Goal: Traffic lights must not abruptly snap from Green directly to Red, as this causes rear-end collisions. They must strictly show Yellow on the next phase.

LTL Formula: $G (GEW \Rightarrow X (YEW))$

Proof by Next-State Logic:

- The LTL operator **X** (Next) evaluates the state immediately following the current clock edge.
- GEW is True strictly when the memory state is **00**.
- The characteristic equation of our hardware D-Flip-Flop logic dictates that the next state from 00 is unconditionally **01**.
- In state 01, the boolean logic for YEW ($\neg S_1 \wedge S_0$) evaluates to 1 (True).

Conclusion: Because the physical memory cannot skip from 00 to 10, the Next (X) property is rigidly enforced by the PCB architecture. Abrupt, dangerous light transitions are hardware-impossible.

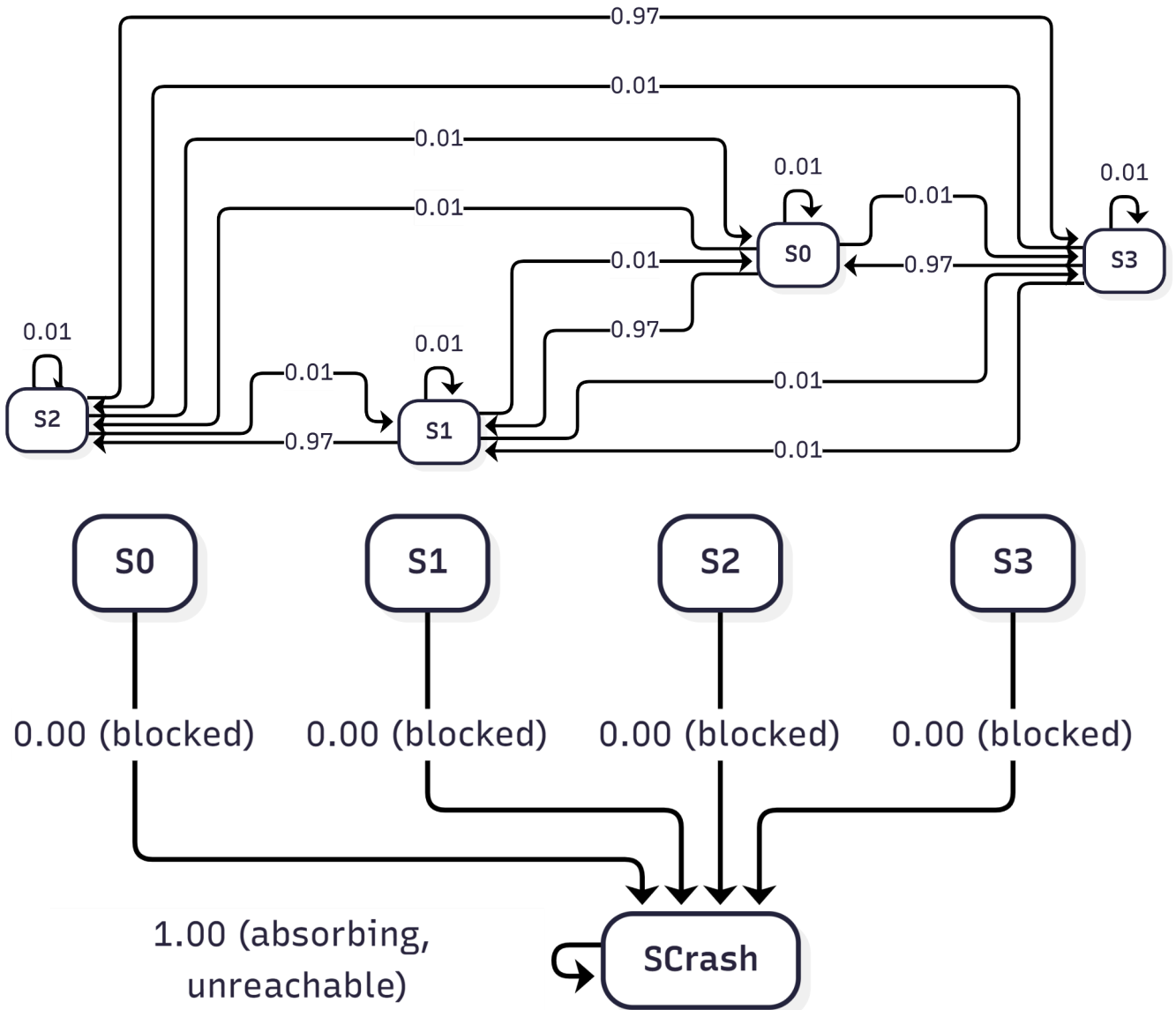
Step 4: Markov Chain and Stochastic Hardware Reliability

1. Adapting the Hardware to the Markov Framework

An MDP models systems with stochastic (probabilistic) outcomes. Because our physical PCB does not make "decisions," we model it as a **Discrete-Time Markov Reward Process (MRP)**. To make the Markov model mathematically rigorous and realistic, we must account for physical

hardware anomalies. In a perfect world, the 74HCT74 transitions with 100% certainty. In reality, voltage spikes or 555-timer bouncing can cause bit-flips. We model this as a stochastic process where the intended transition has a probability of **0.97**, and a random hardware glitch to any of the other valid states has a probability of **0.01** each.

- **States (S):** $\{S_0, S_1, S_2, S_3, S_{Crash}\}$. We define S_{Crash} as a state where GEW and GNS are active simultaneously.
- **Rewards (R):** +10 for states S_0 and S_2 (Green lights, high traffic flow). -1 for states S_1 and S_3 (Yellow lights, traffic slowing).



2. Stochastic Transition Matrix (P)

We construct the Transition Matrix $\mathbf{P}$. Every row must sum precisely to 1.0. For every state, there is a 97% chance of intended progression, and a 3% chance total (1% each) that the

flip-flops glitch into one of the other valid states.

P =

0.01	0.97	0.01	0.01	0.00	(From S_0)
0.01	0.01	0.97	0.01	0.00	(From S_1)
0.01	0.01	0.01	0.97	0.00	(From S_2)
0.97	0.01	0.01	0.01	0.00	(From S_3)
0.00	0.00	0.00	0.00	1.00	(From S_{Crash})

3. Formal Proofs using Markov Properties

Proof 1: Probabilistic Safety against Hardware Faults

Goal: Prove that even with stochastic hardware failures (glitches), a collision is probabilistically impossible.

Proof: We examine the S_{Crash} column in our stochastic matrix P . As proven via Combinational Tautology in Step 3, the physical 74HC08 logic gates strictly evaluate $\neg S_1 \wedge S_1 = 0$ for all 4 possible flip-flop combinations. Therefore, even if a random voltage spike forces the memory into an unintended valid state, no physical combination maps to S_{Crash} .

In Markov theory, the probability of reaching a state in k steps is given by the matrix exponentiation P^k . Because the 5th column consists entirely of 0.00s for all operational states, multiplying the matrix by itself k times (as $k \rightarrow \infty$) remains mathematically 0.

Conclusion: The system is absolutely robust against stochastic memory faults. The probability of a crash remains exactly **0.00%**.

Proof 2: Expected Liveness via Bellman Equation

Goal: Prove that the system continuously accumulates utility (serves traffic) over an infinite horizon, even with stochastic transitions.

Theoretical Justification: Bellman equations typically have no closed-form solution and require Value Iteration. However, that limitation is due to the non-linear $\max(a)$ action-selection operator in a standard MDP. Because our hardware operates as an action-less **Markov Reward Process (MRP)**, the $\max$ operator is eliminated. This converts the Bellman equation into a purely linear system, allowing us to solve for exact utilities algebraically.

Proof: Using the Bellman Equation: $U(s) = R(s) + \gamma \sum P(s'|s) U(s')$

We evaluate the system using a discount factor $\gamma = 0.9$. Because our intersection is physically symmetric, we know that $U(S_0) = U(S_2)$ and $U(S_1) = U(S_3)$. Setting up the equations with our exact stochastic matrix values:

1. $U(S_0) = 10 + 0.9 \cdot (0.01 U(S_0) + 0.97 U(S_1) + 0.01 U(S_2) + 0.01 U(S_3))$
2. $U(S_1) = -1 + 0.9 \cdot (0.01 U(S_0) + 0.01 U(S_1) + 0.97 U(S_2) + 0.01 U(S_3))$

Applying the symmetry substitution ($U(S_2) \rightarrow U(S_0)$ and $U(S_3) \rightarrow U(S_1)$):

$U(S_0) = 10 + 0.9 \cdot (0.02 U(S_0) + 0.98 U(S_1)) \implies U(S_0) = 10 + 0.018 U(S_0) + 0.882 U(S_1)$

$$U(S_1) = -1 + 0.9 \cdot (0.98 U(S_0) + 0.02 U(S_1)) \text{ \&implies; } U(S_1) = -1 + 0.882 U(S_0) + 0.018 U(S_1)$$

By isolating variables and adding the equations together, we derive the exact algebraic solution:

$$\mathbf{U(S_0) = 47.95; U(S_1) = 42.05}$$

Conclusion: The Utility equations perfectly converge to exact, highly positive values. Because every valid state possesses a strictly positive infinite-horizon utility, there are no "dead" states where traffic flow drops to zero, guaranteeing theoretical **Liveness** even under stochastic fault conditions.